

数据库系统

胡 敏 教授 博士生导师
合肥工业大学 计算机与信息学院
jsjxhumin@hfut.edu.cn

第5章 数据库完整性



厚德 笃学 崇实 尚新

第五章 数据库完整性

5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名子句

*5.5 域中的完整性限制

5.6 断言

5.7 触发器

5.8 小结



为什么要显式命名数据库约束？

默认情况下，数据库会生成不直观的系统名（如`students_pkey`），显式命名能带来以下核心价值：

■ 可读性强

好的命名（如`chk_salary_positive`）能清晰表达约束意图，见名知意。

■ 易于管理与维护

修改或删除特定约束时，可通过名称精确操作，无需猜测系统生成的随机标识。

■ 错误排查效率高

违反约束时，错误信息会直接显示约束名称，帮助开发人员快速定位问题根源。



5.4 完整性约束命名子句

1.完整性约束命名子句

CONSTRAINT <完整性约束条件名><完整性约束条件>

或

CONSTRAINT <完整性约束条件名>

[**PRIMARY KEY**短语 | **FOREIGN KEY**短语 |
CHECK短语| **NOT NULL** | **UNIQUE**] ;



完整性约束命名子句（续）

[例5.9]建立学生登记表Student，要求学号在90000~99999之间，姓名不能取空值，年龄小于30，性别只能是“男”或“女”。

```
CREATE TABLE Student
( Sno NUMERIC(6)
  CONSTRAINT C1 CHECK (Sno BETWEEN 90000 AND 99999),
  Sname CHAR(20)
  CONSTRAINT C2 NOT NULL,
  Sage NUMERIC(3)
  CONSTRAINT C3 CHECK (Sage < 30),
  Ssex CHAR(2)
  CONSTRAINT C4 CHECK (Ssex IN ( '男','女')),
  CONSTRAINT StudentKey PRIMARY KEY(Sno)
);
```

- ✓ 在Student表上建立了5个约束条件，包括主码约束（命名为StudentKey）以及C1、C2、C3、C4四个列级约束。



完整性约束命名子句（续）

[例5.10]建立教师表TEACHER，要求每个教师的应发工资不低于3000元。应发工资是工资列Sal与扣除项Deduct之和。

```
CREATE TABLE TEACHER
(  Eno  NUMERIC(4) PRIMARY KEY /*在列级定义主码*/
   Ename CHAR(10),
   Job   CHAR(8),
   Sal   NUMERIC(7,2),
   Deduct NUMERIC(7,2),
   Deptno NUMERIC(2),
   CONSTRAINT TEACHERFKKey FOREIGN KEY (Deptno)
REFERENCES DEPT(Deptno),
   CONSTRAINT C1 CHECK (Sal + Deduct >= 3000)
);
```



完整性约束命名子句（续）

2. 修改表中的完整性限制

➤ 使用ALTER TABLE语句修改表中的完整性限制

[例5.11]去掉例5.9 Student表中对性别的限制。

```
ALTER TABLE Student  
DROP CONSTRAINT C4;
```



完整性约束命名子句（续）

[例5.12] 修改表Student中的约束条件，要求学号改为在900000~999999之间，年龄由小于30改为小于40

➤ 可以先删除原来的约束条件，再增加新的约束条件

```
ALTER TABLE Student
```

```
DROP CONSTRAINT C1;
```

```
ALTER TABLE Student
```

```
ADD CONSTRAINT C1 CHECK (Sno BETWEEN 900000 AND 999999),
```

```
ALTER TABLE Student
```

```
DROP CONSTRAINT C3;
```

```
ALTER TABLE Student
```

```
ADD CONSTRAINT C3 CHECK(Sage < 40);
```



第五章 数据库完整性

5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名字句

***5.5 域中的完整性限制**

5.6 断言

5.7 触发器

5.8 小结



5.5 域 (Domain)完整性约束

■ **域：**域是用户自定义的数据类型，基于基本类型并附加约束（如**CHECK**）。它封装了数据类型与规则，可被多表列复用。

■ 域的核心优势

- **代码复用：**避免在多表中重复编写相同的**CHECK**约束逻辑。
- **集中管理：**业务规则变更时，仅需修改域定义，所有关联列自动继承新约束，维护成本极低。

💡 举例：定义 *GenderType* 域限制为 'M'/'F'，所有性别字段直接引用，一劳永逸。



CREATE DOMAIN域完整性定义

■ 域完整性与 CREATE DOMAIN

通过定义用户自定义的域（数据类型），限制字段取值范围，实现域完整性约束。

■ 语法结构

CREATE DOMAIN 域名 **AS** 基本类型

[DEFAULT 默认值]

[CONSTRAINT 约束名 CHECK (条件)]

注：VALUE 关键字代表域中被检查的当前值。

示例

-- 1. 创建性别域 (仅允许M/F/男/女)

CREATE

DOMAIN GenderType **AS** CHAR(1)

CHECK(VALUE IN ('M','F','男','女'));

-- 2. 创建百分比域 (0-100之间)

CREATE

DOMAIN Percentage **AS** NUMERIC(5,2)

CHECK(VALUE BETWEEN 0 AND 100);



域的应用示例：自定义类型复用

■ 核心场景：表定义中的域复用

通过定义 *GenderType* 和 *Percentage* 自定义域，在创建教师表时直接引用，实现约束的统一管理与复用。

➤ 步骤1：创建表并关联域

在 **CREATE TABLE** 语句中，将字段类型指定为已定义的域名称。

➤ 步骤2：约束生效验证

插入违反域约束的数据（如性别为 'X'）时，数据库会直接抛出错误，拒绝非法数据写入。

-- 1. 创建教师表(使用自定义域)

```
CREATE TABLE Teachers (  
    teacher_id INT PRIMARY KEY,  
    gender GenderType, -- 引用域类型  
    pass_rate Percentage  
);
```

-- 2. 尝试插入非法数据

```
INSERT INTO Teachers(...)  
    VALUES(201, '李教授', 'X');
```

>> 错误：违反域约束



第五章 数据库完整性

5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名字句

*5.5 域中的完整性限制

*5.6 断言

5.7 触发器

5.8 小结



什么是断言 (Assertion)

■ **断言：** 断言是一种更高级的完整性约束，可定义涉及一个或多个表的复杂条件。它与仅作用于单表的 *CHECK* 约束不同，能够对整个数据库的状态进行全局约束。

■ 断言特点

- **全局性：** 断言是独立的数据库对象，不依附于任何特定的表，作用范围覆盖整个数据库。
- **强大性：** 支持表达跨表关联、聚合计算等复杂规则，突破了 *CHECK* 约束的限制。
- **自动检查：** 当任何可能影响断言条件的表执行 *INSERT*、*UPDATE*、*DELETE* 操作时，数据库系统会自动触发断言检查，确保数据合规。

核心价值： 断言是数据库完整性约束中的“大杀器”，确保了跨表复杂逻辑的数据一致性。



断言

- 断言是一个谓词，它表达了我们希望数据库总能满足的一个条件。域约束和参照完整性约束是断言的特殊形式。
- SQL中，可以使用 **CREATE ASSERTION**语句，通过声明性断言来指定更具一般性的约束。
- 可以定义涉及多个表的或聚集操作的比较复杂的完整性约束。
- 断言创建以后，任何对断言中所涉及的关系的操作都会触发关系数据库管理系统对断言的检查，任何使断言不为真值的操作都会被拒绝执行
- SQL Server 2008 及之前版本不支持“断言”
- MySQL 目前不支持“断言”。



断言（续）

1. 创建断言的语句格式

- **CREATE ASSERTION**<断言名><**CHECK** 子句>
- 每个断言都被赋予一个名字，<**CHECK** 子句>中的约束条件与**WHERE**子句的条件表达式类似。

[例5.18] 限制数据库课程最多**60**名学生选修

CREATE ASSERTION ASSE_SC_DB_NUM

CHECK (60 >= (select count(*)

/*此断言的谓词涉及聚集操作count的SQL语句*/

From Course,SC

Where SC.Cno=Course.Cno and

Course.Cname ='数据库')

);



断言（续）

[例5.19]限制每一门课程最多60名学生选修

```
CREATE ASSERTION ASSE_SC_CNUM1
```

```
CHECK(60 >= ALL (SELECT count(*) FROM SC GROUP by cno) );
```

/*此断言的谓词，涉及聚集操作count 和分组函数group by 的SQL语句*/

[例5.20]限制每个学期每一门课程最多60名学生选修

首先需要修改SC表的模式，增加一个“学期（TERM）”属性

```
ALTER TABLE SC ADD TERM DATE;
```

然后，定义断言：

```
CREATE ASSERTION ASSE_SC_CNUM2
```

```
CHECK(60 >= ALL (SELECT count(*)
```

```
FROM SC GROUP by cno,TERM) );
```



断言（续）

2. 删除断言的语句格式为

- **DROP ASSERTION** <断言名>;
- 如果断言很复杂，则系统在检测和维护断言的开销较高，这是在使用断言时应该注意的



断言（续）

■ 断言的利弊：

- 能够保证数据库的一致性；
- 如果断言很复杂，则系统在检测和维护断言的开销较高，这是在使用断言时应该注意的

如：每位教师不能在同一个学期的同一时间段在两个不同的教室上课”



第五章 数据库完整性

5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名字句

*5.5 域中的完整性限制

5.6 断言

5.7 触发器

5.8 小结



触发器

- 触发器（Trigger）是用户定义在关系表上的一类由事件驱动（包括插入、删除和更新）的特殊过程。
 - 触发器保存在数据库服务器中
 - 任何用户对表的增、删、改操作均由服务器自动激活相应的触发器
 - 触发器可以实施更为复杂的检查和操作，具有更精细和更强大的数据控制能力。
 - 检测和维护触发器需要很大的开销，降低了数据库增、删、改的效率！
- **Create Table**中的表约束和列约束基本上都是静态的约束，也基本上都是对单列 或单一元组的约束(尽管有参照完整性)，为实现动态约束以及多个元组之间的完整性 约束，就需要触发器技术 **Trigger**
- **Trigger**是一种过程完整性约束(相比之下，**Create Table**中定义的都是非过程性约束)，是一段程序，该程序可以在特定的时刻被自动触发执行，比如在一次更新操作之前执行，或在更新操作之后执行。



触发器

■ 触发器的作用

- 防止非法的数据库操纵、维护数据库安全
- 对数据库的操作进行审计，存储历史数据
- 完成数据库初始化处理
- 控制数据库的数据完整性
- 进行相关数据的修改
- 完成数据复制
- 自动完成数据库统计计算
- 限制数据库操作的时间、权限等，控制实体的安全性。



触发器

■ 设置触发器机制必须满足的两个条件：

- ① 指明什么条件下触发器被执行，即触发条件；
- ② 指明触发器执行的动作是什么，即触发什么。



5.7 触发器

5.7.1 定义触发器

5.7.2 激活触发器

5.7.3 删除触发器



5.7.1 定义触发器

■ CREATE TRIGGER语法格式

CREATE TRIGGER <触发器名>

{BEFORE | AFTER} <触发事件> ON <表名>

REFERENCING NEW|OLD ROW AS<变量>

FOR EACH {ROW | STATEMENT}

[WHEN <触发条件>]<触发动作体>

触发器又叫做**事件-条件-动作**（**event-condition-action**）规则。

当特定的系统事件发生时，对规则的条件进行检查，如果条件成立则执行规则中的动作，否则不执行该动作。规则中的动作体可以很复杂，通常是一段**SQL存储过程**。



定义触发器（续）

■ 定义触发器的语法说明:

- 1) 创建者: 表的拥有者
- 2) 触发器名: 可以（不）包含模式名, 触发器名和表名必须在同一模式下
- 3) 表名: 触发器的目标表（基本表）
- 4) 触发事件: **INSERT、DELETE、UPDATE**（OF<触发列, ...>）
- 5) 触发器类型
 - 行级触发器（**FOR EACH ROW**）
 - 语句级触发器（**FOR EACH STATEMENT**）
- 6) 触发的时机: **AFTER, BEFORE**
- 7) 触发条件: **WHEN**



定义触发器（续）

■ 定义触发器的语法说明:

[例]在TEACHER表上创建一个AFTER UPDATE触发器，触发事件是UPDATE语句:

```
UPDATE TEACHER SET Deptno=5;
```

- 假设表TEACHER有1000行
- ✓ 如果是语句级触发器，那么执行完该语句后，触发动作只发生一次
- ✓ 如果是行级触发器，触发动作将执行1000次



定义触发器（续）

8) 触发动作体

- 触发动作体可以是一个匿名PL/SQL（Procedure Language/SQL）过程块，也可以是对已创建存储过程的调用
- 如果是行级触发器，用户都可以在过程体中使用NEW（INSERTED）和OLD（DELETED）引用事件之后的新值和事件之前的旧值
- 如果是语句级触发器，则不能在触发动作体中使用NEW（INSERT）或OLD（DELETE）进行引用
- 如果触发动作体执行失败，激活触发器的事件就会终止执行，触发器的目标表或触发器可能影响的其他对象不发生任何变化

注意：不同的RDBMS产品触发器语法各不相同



定义触发器（续）

- 一个数据库系统中有两个虚拟表用于存储在表中记录改动的信息
- 理解触发器里面的两个临时的表
 - Oracle（**PL/SQL**）：New , Old
old 与new分别表示触发事件的表“旧的一条记录”和“新的一条记录”。
 - SQL Server（**T-SQL**）：Deleted , Inserted
Deleted 与Inserted分别表示触发事件的表“旧的一条记录”和“新的一条记录”。



定义触发器（续）

[例5.21]当对表SC的Grade属性进行修改时，若分数增加了10%则将此次操作记录到下面表中：

SC_U (Sno,Cno,Oldgrade,Newgrade)

其中Oldgrade是修改前的分数，Newgrade是修改后的分数。

```
CREATE TRIGGER SC_T  
AFTER UPDATE OF Grade ON SC  
REFERENCING
```

OLD row AS OldTuple,

NEW row AS NewTuple

FOR EACH ROW

WHEN (NewTuple.Grade >= 1.1*OldTuple.Grade)

INSERT INTO SC_U(Sno,Cno,OldGrade,NewGrade)

VALUES(OldTuple.Sno,OldTuple.Cno,OldTuple.Grade,NewTuple.Grade)

注：不同DBMS 定义格式也不同

如：MySQL

```
CREATE TRIGGER SC_T  
AFTER UPDATE ON SC  
FOR EACH ROW  
IF (New.Grade >= 1.1*Old.Grade) THEN  
INSERT INTO  
SC_U(Sno,Cno,OldGrade,NewGrade)  
VALUES(Old.Sno,Old.Cno,Old.Grade,New.Grade);  
end IF;
```

Sno	Cno	OldGrade	NewGrade
202215122	2	90	100
202215122	3	66	80



定义触发器（续）

[例5.22] 将每次对表Student的插入操作所增加的学生个数记录到表StudentInsertLog中。

```
CREATE TRIGGER Student_Count  
AFTER INSERT ON Student
```

```
/*指明触发器激活的时间是在执行INSERT后*/
```

```
REFERENCING
```

```
NEW TABLE AS DELTA
```

```
FOR EACH STATEMENT
```

```
/*语句级触发器, 即执行完INSERT语句后下面的触发动作体才执行一次*/
```

```
INSERT INTO StudentInsertLog (Numbers)
```

```
SELECT COUNT(*) FROM DELTA
```



定义触发器（续）

例. 学生成绩不低于60分,低于60分自动赋为60分

SQL Server

```
Create Trigger chggrade
on SC for insert
as
update SC set grade=60
where exists (select * from inserted
where inserted.Sno=SC.Sno and
inserted.Cno=SC.Cno
and inserted.grade < 60 )
```



定义触发器（续）

[例5.23] 定义一个BEFORE行级触发器，为教师表Teacher定义完整性规则“教授的工资不得低于4000元，如果低于4000元，自动改为4000元”。

```
CREATE TRIGGER Insert_Or_Update_Sal
BEFORE INSERT OR UPDATE ON Teacher
    /*触发事件是插入或更新操作*/
FOR EACH ROW    /*行级触发器*/
BEGIN
    /*定义触发动作体，是PL/SQL过程块*/
    IF (new.Job='教授') AND (new.Sal < 4000)
    THEN new.Sal :=4000;
    END IF;
END;
```



5.7 触发器

5.7.1 定义触发器

5.7.2 激活触发器

5.7.3 删除触发器



5.7.2 激活触发器

- 触发器的执行，是由触发事件激活的，并由数据库服务器自动执行
- 一个数据表上可能定义了多个触发器，遵循如下的执行顺序：
 - (1) 执行该表上的BEFORE触发器;
 - (2) 激活触发器的SQL语句;
 - (3) 执行该表上的AFTER触发器。



5.7 触发器

5.7.1 定义触发器

5.7.2 激活触发器

5.7.3 删除触发器



5.7.3 删除触发器

- 删除触发器的SQL语法:

DROP TRIGGER <触发器名> ON <表名>;

- 触发器必须是一个已经创建的触发器，并且只能由具有相应权限的用户删除。

[例] 删除成绩表SC上的触发器chggrade 。

DROP TRIGGER chggrade ON SC;



第五章 数据库完整性

5.1 实体完整性

5.2 参照完整性

5.3 用户定义的完整性

5.4 完整性约束命名字句

*5.5 域中的完整性限制

5.6 断言

5.7 触发器

5.8 小结



5.8 小结

- 数据库的完整性是为了保证数据库中存储的数据是正确的
- 关系数据库管理系统完整性实现的机制
 - 完整性约束定义机制
 - 完整性检查机制
- 违背完整性约束条件时关系数据库管理系统应采取的动作



完整性约束的检查顺序（优先级）

① 域约束

② DEFAULT

③ NOT NULL

④ CHECK

⑤ 对有FOREIGN KEY约束的参照表的检查

⑥ PRIMARY KEY

⑦ 触发器

■ 完整性机制的实施会极大地影响系统性能

■ 不同的数据库产品对完整性的支持策略和支持程度是不同的

➤ 许多数据库管理系统对完整性机制的支持比对安全性的支持要晚得多也弱得多

➤ 数据库厂商对完整性的支持越来越好，不仅在能保证实体完整性和参照完整性而且能在DBMS核心定义、检查和保证用户定义的完整性约束条件



作业

■ P173: 6、7

■ 实验



To be continued
Practice makes perfect!



厚德

笃学

崇实

尚新